

# NUCLEO COGNITIVO DA AURORA SIGER (NCAS)

## *Regras Logicas e Simplificacao Booleana*

Este documento apresenta as regras logicas implementadas no sistema NCAS, as expressoes booleanas originais, suas simplificacoes e a justificativa de que a simplificacao mantem o mesmo resultado logico.

### **Regra 1 - Geracao de alerta critico (Teorema da Simplificacao)**

Contexto: o sistema deve gerar um alerta sempre que houver falha em um modulo, independentemente de essa falha ser classificada como critica ou nao critica.

Expressao original:

```
ALERTA = (FALHA AND CRITICO) OR (FALHA AND NOT CRITICO)
```

Aplicando o Teorema da Simplificacao (tambem chamado de Absorcao), que estabelece que  $A.B + A.B' = A$ , tomando  $A = FALHA$  e  $B = CRITICO$ , temos:

```
FALHA.CRITICO + FALHA.(NOT CRITICO) = FALHA
```

Expressao simplificada:

```
ALERTA = FALHA
```

Justificativa: nos dois unicos casos possiveis para CRITICO (verdadeiro ou falso), a expressao original so resulta em verdadeiro quando FALHA e verdadeira. Quando FALHA e falsa, ambos os termos da disjuncao (OR) sao falsos, e quando FALHA e verdadeira, pelo menos um dos termos sera verdadeiro (dependendo de CRITICO), tornando o OR sempre verdadeiro. Portanto, o valor de CRITICO nao interfere no resultado final, e a expressao pode ser reduzida apenas a FALHA, sem alterar a tabela-verdade original. Essa simplificacao foi implementada na funcao verificar\_alerta() do codigo\_fonte.py, inclusive com uma verificacao (assert) que compara o resultado da expressao original com o da expressao simplificada para todos os casos testados.

### **Regra 2 - Liberacao de consulta ao sistema**

Contexto: uma consulta so deve ser liberada se o usuario estiver autorizado e o modulo consultado estiver ativo.

Expressao (conjuncao simples, sem necessidade de simplificacao adicional):

```
LIBERAR_CONSULTA = AUTORIZADO AND MODULO_ATIVO
```

Essa regra representa a algebra booleana basica de uma porta AND: o resultado so e verdadeiro quando ambas as condicoes (autorizacao e modulo ativo) sao verdadeiras simultaneamente.

### **Regra 3 - Bloqueio de operacao (Teorema de De Morgan)**

Contexto: o sistema deve bloquear uma operacao sempre que a condicao de liberacao (Regra 2) nao for satisfeita.

Expressao original:

```
BLOQUEAR = NOT (AUTORIZADO AND MODULO_ATIVO)
```

Aplicando o Teorema de De Morgan, que estabelece que  $\text{NOT } (A \text{ AND } B) = (\text{NOT } A) \text{ OR } (\text{NOT } B)$ , obtemos a forma equivalente:

```
BLOQUEAR = (NOT AUTORIZADO) OR (NOT MODULO_ATIVO)
```

Justificativa: a expressao com De Morgan e logicamente equivalente a negacao da conjuncao original, pois basta que uma das duas condicoes falhe (usuario nao autorizado ou modulo inativo) para que a operacao deva ser bloqueada. Essa forma e mais direta de interpretar no codigo, pois separa claramente as duas causas possiveis do bloqueio, sem alterar o resultado logico da expressao original. A funcao `bloquear_operacao()` do `codigo_fonte.py` implementa ambas as formas e usa um `assert` para comprovar a equivalencia.

Conclusao: as tres regras logicas foram implementadas em Python utilizando operadores booleanos nativos (`and`, `or`, `not`) e validadas por meio de `asserts` que comparam a forma original com a forma simplificada, comprovando que as simplificacoes preservam o comportamento logico do sistema.